

Start coding or generate with AI.

Scikit-learn (also known as sklearn) is a widely-used open-source Python library for machine learning.

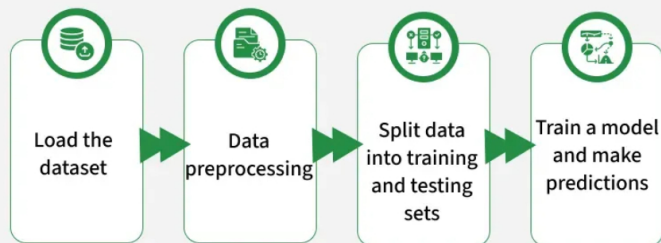
It builds on other scientific libraries like NumPy, SciPy and Matplotlib to provide efficient tools for predictive data analysis and data mining.

It offers a consistent and simple interface for a range of supervised and unsupervised learning algorithms, including classification, regression, clustering, dimensionality reduction, model selection and preprocessing.

Scikit-learn simplifies machine learning with pre-built tools and algorithms.



Steps to Build a Model with Scikit-learn



Why Choose Scikit-learn?

- Scikit-learn offers tools for preprocessing, training and evaluation.
- It supports easy algorithm switching and hyperparameter tuning.



Installing Scikit-learn in Google Colab

While Scikit-learn is usually pre-installed in Google Colab, you might want to manually install it to ensure you have the latest version or if it's not available. Follow these steps:

Step 1: Install Scikit-learn

```
!pip install -U scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Collecting scikit-learn
  Downloading scikit_learn-1.8.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (11 kB)
Requirement already satisfied: numpy>=1.24.1 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy<1.16.0, >=1.12.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.15.0)
```

```
Requirement already satisfied: scipy>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.10.0)
Requirement already satisfied: joblib>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Downloading scikit_learn-1.8.0-cp312-cp312-manylinux_2_28_x86_64.manylinux_2_28_x86_64.whl (8.9 MB)
8.9/8.9 MB 42.1 MB/s eta 0:00:00

Installing collected packages: scikit-learn
Attempting uninstall: scikit-learn
Found existing installation: scikit-learn 1.6.1
Uninstalling scikit-learn-1.6.1:
Successfully uninstalled scikit-learn-1.6.1
Successfully installed scikit-learn-1.8.0
```

Step 2: Verify the Installation

To verify that Scikit-learn has been installed correctly, you can import it and check its version with the following code:

```
import sklearn
print("Scikit-learn version:", sklearn.__version__)
```

Scikit-learn version: 1.8.0

Double-click (or enter) to edit

Scikit-learn Library?

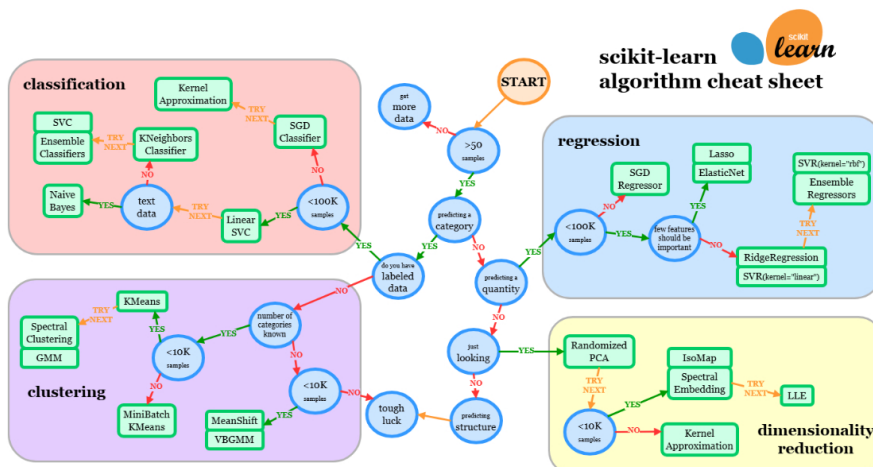
Scikit-learn is an open-source machine learning library that provides simple and efficient tools for data analysis and modeling. It is built on NumPy, SciPy, and Matplotlib, making it a powerful tool for tasks like classification, regression, clustering, and dimensionality reduction.

Classification: Classification involves teaching a computer to categorize things. For example, a model could be built to determine whether an email is spam or not.

Regression: Regression predicting numbers based on other numbers. For instance, a model could predict house prices using factors like location, size, and age.

Clustering: Clustering involves finding patterns in data and grouping similar items together. For example, customers could be segmented into different groups based on their shopping habits.

Dimensionality Reduction: Dimensionality reduction helps focus on essential data parts while discarding noise. This is useful when dealing with a lot of data that isn't all relevant.



1 Classification

Used when output is **categorical** (Yes/No, Disease/No Disease)

Examples:

- Spam vs Non-Spam Email
- Fake vs Real News
- Disease: Present / Absent

Algorithms:

- Logistic Regression
- KNN
- Naive Bayes
- Decision Tree
- SVM

2 Regression

Used when output is **numeric**

Examples:

- House price prediction
- Crop yield prediction
- Temperature prediction

Algorithms:

- Linear Regression
- Polynomial Regression
- Ridge & Lasso

3 Clustering

Used when **no labels are given**

Examples:

- Customer grouping
- Patient similarity
- Student performance groups

Algorithms:

- K-Means
- Hierarchical Clustering
- DBSCAN

4 Model Evaluation

Helps students **check model performance**

Metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Features of Scikit-Learn

Scikit-learn is indeed a versatile tool for machine learning tasks, offering a wide range of features to address various aspects of the data science pipeline. let's examine prime key features of scikit-learn:

Supervised Learning

Classification: Algorithms for predicting categorical labels, including logistic regression, decision trees, random forests, support vector machines (SVMs) and gradient boosting.

Regression: Algorithms for predicting continuous outputs, including linear regression, support vector regression, and decision tree regression.

Unsupervised Learning

Clustering: Techniques for grouping data points into similar clusters, including K-means clustering, DBSCAN, and hierarchical clustering.

Dimensionality Reduction: Methods for reducing the number of features in your data, such as principal component analysis (PCA).

Data Preprocessing

Data Splitting: Functions to split your data into training and testing sets for model evaluation.

Feature Scaling: Techniques for normalizing the scale of your features.

Feature Selection: Methods to identify and select the most relevant features for your model.

Feature Extraction: Tools to create new features from existing ones, such as text vectorization for natural language processing tasks.

Model Evaluation

Metrics: Functions to calculate performance metrics like accuracy, precision, recall, and F1-score for classification models, and mean squared error (MSE) for regression models.

Model Selection: Tools for selecting the best model and hyperparameters through techniques like grid search and randomized search.

Additional Features

Inbuilt datasets: Scikit-learn provides a variety of sample datasets for experimentation and learning purposes.

Easy to Use API: Scikit-learn is known for its consistent and user-friendly API, making it accessible to both beginners and experienced data scientists.

Open Source: Scikit-learn is an open-source library with a large and active community, ensuring continuous development and support.

Data Normalization with Python Scikit-Learn

Data normalization is a crucial step in machine learning and data science. It involves transforming features to similar scales to improve the performance and stability of machine learning models.

Python's Scikit-Learn library provides several techniques for data normalization, which are essential for ensuring that models are not biased towards features with large ranges.

Feature Engineering?

Feature Engineering is the process of creating, transforming, extracting, selecting, and scaling features to improve the performance of a machine learning model.

Good features = Good model

Feature Creation

Definition

Feature creation involves generating new features from existing data using domain knowledge or observed patterns.

Types of Feature Creation

a) Domain-Specific Features

Created using subject or industry knowledge.

Example (Students Dataset):

Attendance percentage

Pass/Fail based on marks

Grade (A/B/C)

📌 Based on rules & experience

b) Data-Driven Features

Created by analyzing patterns in data.

Example:

Average marks across subjects

Growth in marks over semesters

📌 Based on data trends

📌 Based on data types

✅ c) Synthetic Features

Created by combining existing features.

Example:

Total marks = Math + Science + English

Marks per study hour

📌 Helps models capture relationships

2 Feature Transformation

💠 Definition

Feature transformation modifies existing features to make them more suitable for learning.

💠 Types of Feature Transformation

✅ a) Normalization & Scaling

Adjusts feature values to a common scale.

Why needed?

Prevents bias due to different ranges

Improves model convergence

✅ b) Encoding (Categorical → Numerical)

Used to convert non-numeric data into numbers.

Example:

Gender → Male = 0, Female = 1

Grade → One-Hot Encoding

📌 ML models work only with numbers

✅ c) Mathematical Transformations

Applied to reduce skewness.

Example:

Log transformation

Square root transformation

📌 Improves learning for skewed data

3 Feature Extraction

💠 Definition

Feature extraction reduces data complexity by extracting important information from original features.

💠 Methods of Feature Extraction

✅ a) Dimensionality Reduction

Reduces number of features while preserving information.

Example:

PCA (Principal Component Analysis)

📌 Reduces overfitting & computation

✅ b) Aggregation & Combination

Combines multiple features into one.

Example:

Average marks

Weekly → Monthly averages

📌 Simplifies the model

4 Feature Selection

◆ Definition

Feature selection chooses the most relevant features and removes unnecessary ones.

◆ Types of Feature Selection

✅ a) Filter Methods

Based on statistical measures.

Examples:

Correlation

Chi-square test

📌 Fast & model-independent

✅ b) Wrapper Methods

Use model performance to select features.

Examples:

Forward selection

Backward elimination

📌 More accurate but computationally expensive

✅ c) Embedded Methods

Feature selection happens during model training.

Examples:

Lasso Regression

Decision Tree feature importance

📌 Efficient and commonly used

5 Feature Scaling

◆ Definition

Feature scaling ensures that all features contribute equally to the model.

◆ Types of Feature Scaling

✅ a) Min-Max Scaling

Rescales values to a fixed range (0–1).

Formula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

📌 Used in distance-based models

~ ✅ b) Standard Scaling (Z-Score Normalization)

Transforms data to:

Mean = 0

Standard Deviation = 1

Formula:

$$Z = \frac{X - \mu}{\sigma}$$

🔗 Used in SVM, Logistic Regression, PCA

Step	Purpose	Example
Feature Creation	Add new info	Total marks
Feature Transformation	Improve learning	Encoding
Feature Extraction	Reduce complexity	PCA
Feature Selection	Keep useful features	Correlation
Feature Scaling	Equal importance	StandardScaler

Students Performance Analysis

Step 0: Original Students Dataset (Raw Data)

Assume we collect this data from a class:

Student	Study Hours	Attendance (%)	Math	Science	English
S1	2	60	40	42	45
S2	4	70	55	58	60
S3	6	80	70	65	68
S4	8	90	85	78	80
S5	10	95	95	92	90

👉 This is raw data

👉 ML cannot learn well directly from this

So we apply Feature Engineering 📌

1 Feature Creation (Creating New Information)

💠 What we do?

We create new useful columns from existing data.

✅ Examples (Very Easy)

💠 Total Marks

💠 Total Marks

```
ini
Total = Math + Science + English
```

💠 Average Marks

```
ini
Average = Total / 3
```

💠 Performance Label (Domain Knowledge)

```
nginx
Average ≥ 60 → Good
Average < 60 → Poor
```

📊 After Feature Creation

Student	Total	Average	Performance
S1	127	42.3	Poor

S2	173	57.7	Poor
S3	203	67.7	Good
S4	243	81.0	Good
S5	277	92.3	Good

🧠 Student understanding line:

“We created new columns to better describe student performance.”

2 Feature Transformation (Making Data ML-Friendly)

◆ Problem

ML models cannot understand text.

◆ Transform Categorical → Numerical

Performance:

Poor → 0

Good → 1

Now:

Student	Performance
S1	0
S2	0
S3	1
S4	1
S5	1

Meaning:

“We converted labels into numbers.”

3 Feature Extraction (Reducing Complexity)

◆ Problem

We have many marks columns:

Math

Science

English

◆ Extract One Meaningful Feature

👉 Use Average Marks instead of all subjects.

📌 This:

Reduces complexity

Makes model faster

Avoids overfitting

Start coding or generate with AI.

🧠 Meaning:

“Instead of many subjects, we summarize performance.”

4 Feature Selection (Choosing Important Features)

What finding

Do we need all features?

Let's analyze importance:

Let's analyze importance:

Feature	Useful?
Study Hours	✔ Yes
Attendance	✔ Yes
Math	✗ Redundant (already in average)
Science	✗
English	✗

✔ Selected Features

CSS

Study Hours
Attendance
Average Marks

💡 Meaning:

"We keep only features that really affect performance."

5 Feature Scaling

◆ Problem

Feature	Range
Study Hours	2-10
Attendance	60-95
Average Marks	40-92

Different ranges ✗ → model biased

◆ Apply Scaling (Conceptually)

After **Standard Scaling**:

- Mean = 0
- Same importance

💡 Student line:

"Scaling makes all features equally important."

FINAL DATA (What ML Actually Plays here)

Study Hours	Attendance	Avg Marks	Performance
-1.2	-1.1	-1.3	0
-0.6	-0.5	-0.6	0
0.0	0.0	0.1	1
0.7	0.8	0.9	1
1.1	1.2	1.4	1

Raw Data

→ Create Features
→ Transform
→ Extract

- Select
- Scale
- ML Model

Feature creation: Adding new meaningful columns

Feature transformation: Converting data into usable form

Feature extraction: Reducing dimensions

Feature selection: Choosing relevant features

Feature scaling: Normalizing feature ranges

Evaluation Metrics in Machine Learning

🎓 Dataset Context (Used Everywhere)

Assume we have a Students Performance Model.

Two Problems:

Classification → Pass / Fail

Regression → Predict Marks

Clustering → Group students by performance

PART 1: CLASSIFICATION METRICS

(Pass / Fail Prediction)

Assume:

Pass = Yes

Fail = No

Confusion Matrix Result (Given)

Confusion Matrix Result (Given)

	Predicted No	Predicted Yes
Actual No	50 (TN)	10 (FP)
Actual Yes	5 (FN)	100 (TP)

Total students = 165

1 Accuracy

? What does it tell?

Overall correctness of the model

Formula:

Formula:

$$Accuracy = \frac{TP + TN}{Total}$$

Students Example:

$$Accuracy = \frac{100 + 50}{165} = 0.91 \text{ (91\%)}$$

🧠 Meaning:

"91% students are correctly classified."

⚠️ Problem:

If most students pass → accuracy looks high even if failures are misclassified.

2 Precision

? What does it tell?

When model says "Pass", how often is it correct?

Formula:

$$Precision = \frac{TP}{TP + FP}$$

Students Example:

$$Precision = \frac{100}{100 + 10} = 0.91$$

Meaning:

"Out of all predicted pass students, 91% actually passed."

🚩 Important when false pass is costly.

3 Recall (Sensitivity)

? What does it tell?

How many actual pass students were caught?

Formula:

$$Recall = \frac{TP}{TP + FN}$$

Students Example:

$$Recall = \frac{100}{100 + 5} = 0.95$$

Meaning:

"We caught 95% of all students who actually passed."

🚩 Important when missing pass students is bad.

Double-click (or enter) to edit

Double-click (or enter) to edit

4 F1-Score

? What does it tell?

Balance between Precision and Recall

Formula:

$$F1 = \frac{2PR}{P + R}$$

Students Example:

$$F1 \approx 0.93$$

Meaning:

"Model is good at both identifying pass students and avoiding wrong passes."

↑ ↓ ✎ 🗑️ ⋮

Feature Creation: Generating new features from existing data

Feature Transformation: Modifying features for better learning

Feature Extraction: Reducing data dimensionality

Feature Selection: Choosing relevant features

Feature Scaling: Bringing features to a common scale

- Feature engineering includes creating, transforming, extracting, selecting, and scaling features to improve machine learning model accuracy and efficiency.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Why Data Normalization is Necessary?

Machine learning algorithms are sensitive to the scale of features. Features with large ranges can dominate the model, leading to poor performance and inaccurate results.

Data normalization helps to mitigate this issue by transforming features to a common scale, ensuring that all features contribute equally to the model.

What is Min-Max Normalization?

Min-Max normalization rescales the data between 0 and 1.

- Formula:

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Double-click (or enter) to edit

Why use it?

When feature values have very different ranges

Keeps original data distribution shape

Works well with distance-based algorithms

- New value lies between 0 and 1

◆ When to use?

- KNN
- K-Means
- Neural Networks

◆ Advantage:

- ✓ Simple and fast

◆ Disadvantage:

- ✗ Sensitive to outliers

Z-Score Normalization is a feature scaling technique used in Machine Learning to transform data so that:

Mean = 0

Standard Deviation = 1

It tells us how far a data value is from the average.

Why do we use Z-Score Normalization?

- ✓ Removes scale difference
- ✓ Makes data centered around 0
- ✓ Improves performance of many ML models

When is it used?

Z-Score Normalization is best for:

Logistic Regression

Support Vector Machine (SVM)

Principal Component Analysis (PCA)

Linear Regression

✓ Robust Scaling?

Robust Scaling is a feature scaling technique used in Machine Learning that reduces the effect of outliers.

Instead of using mean and standard deviation, it uses:

Median

Interquartile Range (IQR)

That's why it is called robust.

Simple Meaning (Very Easy Words)

If your data has very large or very small values (outliers)

Min-Max and Z-Score get disturbed ✗

Robust Scaling still works well

Formula

$$X' = \frac{X - \text{Median}}{IQR}$$

Where:

- **Median** = middle value
- **IQR** = Q3 - Q1 (difference between 75% and 25%)

Simple Example

Suppose salaries (₹):

20,000 22,000 23,000 25,000 5,00,000

 ₹5,00,000 is an **outlier**

- Mean gets disturbed ✗
- Median remains stable ✓

So **Robust Scaling** gives better scaled values.

✓ When should students use Robust Scaling?

Best for:

Medical data

Financial data

Sensor data

Income / salary datasets

Method	Uses Mean	Uses Median	Outlier Effect
Min-Max	✗	✗	✗ High
Z-Score	✓	✗	✗ Medium
Robust	✗	✓	✓ Very Low

Easy Trick to Remember

Min-Max → Range based

Z-Score → Mean based

Robust → Outlier safe

Feature selection?

Feature selection is the process of identifying and selecting a subset of relevant features for use in model construction.

The goal is to enhance the model's performance by reducing overfitting, improving accuracy, and reducing training time.

Feature Selection is the process of choosing the most important input features from a dataset and removing unnecessary ones.

👉 In simple words:

Use only useful columns, ignore useless ones.

Suppose you want to predict **student result (Pass/Fail)**.

Available features:

- Age
- Attendance ✓
- Study Hours ✓
- Roll Number ✗
- Phone Number ✗

👉 Feature selection keeps **Attendance & Study Hours** and removes others.

Why is Feature Selection Important?

- ✓ Improves model accuracy
- ✓ Reduces overfitting
- ✓ Decreases training time
- ✓ Makes model simple & interpretable

```
import pandas as pd

data = {
    'Age': [18, 19, 20, 21, 22],
    'Attendance': [75, 85, 60, 90, 95],
    'Study_Hours': [2, 4, 1, 5, 6],
    'Roll_Number': [101, 102, 103, 104, 105],
    'Phone_Number': [9876, 8765, 7654, 6543, 5432],
    'Result': [0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

X = df.drop('Result', axis=1)
y = df['Result']
```

FILTER METHOD – SelectKBest (Statistical Method)

🚀 Uses ANOVA F-test to select best features.

```
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(score_func=f_classif, k=2)
X_filter = selector.fit_transform(X, y)
selected_features = X.columns[selector.get_support()]
print("Selected Features (Filter Method):", selected_features)
```

Selected Features (Filter Method): Index(['Attendance', 'Study_Hours'], dtype='object')

2 WRAPPER METHOD – Recursive Feature Elimination (RFE)

🚀 Uses Logistic Regression to select best features.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
rfe = RFE(model, n_features_to_select=2)
X_wrapper = rfe.fit_transform(X, y)
selected_features = X.columns[rfe.support_]
print("Selected Features (Wrapper Method):", selected_features)
```

Selected Features (Wrapper Method): Index(['Attendance', 'Study_Hours'], dtype='object')

FEATURE SCALING

Feature Scaling is a data preprocessing technique used to bring all numerical features to a similar scale so that no feature dominates others during model training.

Why Feature Scaling is Needed?

In real datasets, different features have different ranges.

Machine learning algorithms may give more importance to large-value features, which leads to poor performance.

Uses of Feature Scaling

- Improves accuracy of ML models
- Speeds up model convergence
- Ensures fair contribution of all features
- Essential for distance-based algorithms
- Makes data suitable for optimization algorithms

Types of Feature Scaling

Min-Max Normalization

🚀 Definition

Min-Max normalization rescales data into a fixed range (0 to 1) using minimum and maximum values.

🚀 Formula

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

🚀 Uses

- When features have **different ranges**
- Neural Networks
- KNN, K-Means

🚀 Advantages

- ✔ Simple to understand
- ✔ Preserves original data distribution
- ✔ Fast computation

🚩 **Disadvantages**

- ✖ Sensitive to outliers
- ✖ New unseen values may go out of range

2 Z-Score Normalization (Standardization)

🚩 **Definition**

Z-Score normalization transforms data to have mean = 0 and standard deviation = 1.

Start coding or [generate](#) with AI.

🚩 **Formula**

$$Z = \frac{X - \mu}{\sigma}$$

🚩 **Uses**

- Logistic Regression
- SVM
- PCA
- Linear Regression

🚩 **Advantages**

- ✔ Handles different units well
- ✔ Suitable for normally distributed data
- ✔ Improves gradient-based learning

🚩 **Disadvantages**

- ✖ Affected by extreme outliers
- ✖ Results include negative values

Start coding or [generate](#) with AI.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Step 1: Create the dataset
data = {
    'Age': [18, 19, 20, 21, 22],
    'Attendance': [75, 85, 60, 90, 95],
    'Study_Hours': [2, 4, 1, 5, 6],
    'Roll_Number': [101, 102, 103, 104, 105],
    'Phone_Number': [9876, 8765, 7654, 6543, 5432]
}

df = pd.DataFrame(data)
# Step 2: Apply Z-Score Normalization (Standardization)
scaler = StandardScaler()
zscore_data = scaler.fit_transform(df)
# Step 3: Convert to DataFrame for readable output
zscore_df = pd.DataFrame(zscore_data, columns=df.columns)
# Step 4: Display result
print(zscore_df)
```

	Age	Attendance	Study_Hours	Roll_Number	Phone_Number
0	-1.414214	-0.483494	-0.862662	-1.414214	1.414214
1	-0.707107	0.322329	0.215666	-0.707107	0.707107
2	0.000000	-1.692228	-1.401826	0.000000	0.000000
3	0.707107	0.725241	0.754829	0.707107	-0.707107
4	1.414214	1.128152	1.293993	1.414214	-1.414214

In Z-Score Normalization:

1.2 Z-Score Normalization

0 → average value

Negative value → below average

Positive value → above average

Bigger number → farther from the mean

Final Comparison

Feature	Min-Max Scaling	Z-Score Normalization
Output Range	0 to 1	No fixed range
Uses	Min & Max	Mean & Std Dev
Negative Values	✗ No	✓ Yes
Sensitive to Outliers	✗ Yes	⚠ Less
Interpretation	Relative position	Distance from mean
Best For	KNN, Neural Nets	SVM, Logistic Reg.

Simple Flow for ML

```
Raw Data
↓
Basic Preprocessing
↓
Feature Selection
↓
Feature Scaling
↓
Train-Test Split
↓
Model Training
```

Double-click (or enter) to edit

3 Robust Scaling

📌 Definition

Robust Scaling uses median and interquartile range (IQR) to scale features and is resistant to outliers.

📌 Formula

📌 Formula

$$X' = \frac{X - \text{Median}}{IQR}$$

📌 Uses

- Financial data
- Medical data
- Data with extreme values

📌 Advantages

- ✓ Best for outlier-affected data
- ✓ Stable and reliable
- ✓ Real-world friendly

📌 Disadvantages

- ✗ Less intuitive than Min-Max
- ✗ No fixed output range

Feature Scaling: Technique to normalize feature values

Min-Max Scaling: Scales data between 0 and 1

Z-Score Scaling: Mean 0 and standard deviation 1

Robust Scaling: Scaling using median and IQR

Double-click (or enter) to edit

Double-click (or enter) to edit

Why Feature Scaling is Needed

Feature Selection	Feature Scaling
Selects features	Scales feature values
Removes columns	Keeps all columns
Improves relevance	Improves magnitude
Example: drop Age	Example: normalize Age

Why Feature Scaling is Needed Practically

Why Feature Scaling is Needed (Student View)

Feature	Range
Age	18 – 22
Attendance	60 – 95
Study Hours	1 – 6
Roll Number	101 – 105
Phone Number	5000 – 9000

✗ Large values dominate small ones

✗ ML models like **KNN, SVM, Logistic Regression** get confused

✓ Feature scaling fixes this

```
import pandas as pd

data = {
    'Age': [18, 19, 20, 21, 22],
    'Attendance': [75, 85, 60, 90, 95],
    'Study_Hours': [2, 4, 1, 5, 6],
    'Roll_Number': [101, 102, 103, 104, 105],
    'Phone_Number': [9876, 8765, 7654, 6543, 5432],
    'Result': [0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

X = df.drop('Result', axis=1)
y = df['Result']
```

1 Min-Max Scaling (0 to 1)

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_minmax = scaler.fit_transform(X)

X_minmax_df = pd.DataFrame(X_minmax, columns=X.columns)
print(X_minmax_df)
```

	Age	Attendance	Study_Hours	Roll_Number	Phone_Number
0	0.00	0.428571	0.2	0.00	1.00
1	0.25	0.714286	0.6	0.25	0.75

2	0.50	0.000000	0.0	0.50	0.50
3	0.75	0.857143	0.8	0.75	0.25
4	1.00	1.000000	1.0	1.00	0.00

What does this result mean?

- 👉 All feature values are now between 0 and 1
- 👉 Relative order of values is preserved
- 👉 No column is removed (only values changed)

This confirms **Min-Max Scaling** is applied correctly ✅

Key Observations

All values are between 0 and 1

- ✓ Maximum value → 1.0
- ✓ Minimum value → 0.0
- ✓ Relative order is preserved
- ✓ No feature dominance remains

⌵ Common Doubts

Q1. Why are some values exactly 0 or 1?

👉 Because they are **minimum or maximum** in that column.

Q2. Why Attendance has decimals like 0.428571?

👉 Because scaling is **ratio-based**, not rounding.

Q3. Does scaling change meaning of data?

👉 ❌ No, only scale changes, not relationship.

⌵ Data Leakage vs Correct Preprocessing

Aim of the Practical

To demonstrate:

What is data leakage

Why it occurs

How to avoid it

using student data (Age, Attendance, Study Hours).

Dataset Used

Feature	Description
Age	Student age
Attendance	Attendance percentage
Study_Hours	Daily study hours
Result	0 = Fail, 1 = Pass

⌵ STEP 1: Import Required Libraries

pandas → for data handling

StandardScaler → for Z-score normalization

LogisticRegression → classification model

accuracy_score → to evaluate performance

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

▼ STEP 2: Create Student Dataset

X → input features

y → target variable (Pass/Fail)

```
data = {
    'Age': [18, 19, 20, 21, 22, 19, 20, 21, 18, 22],
    'Attendance': [75, 85, 60, 90, 95, 88, 65, 92, 70, 98],
    'Study_Hours': [2, 4, 1, 5, 6, 4, 2, 5, 3, 6],
    'Result': [0, 1, 0, 1, 1, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

X = df[['Age', 'Attendance', 'Study_Hours']]
y = df['Result']
```

▼ CASE 1: WRONG METHOD (DATA LEAKAGE)

◆ STEP 3A: Apply Scaling Before Train-Test Split (WRONG)

```
scaler = StandardScaler()
X_scaled_wrong = scaler.fit_transform(X)  # ❌ leakage happens here
```

▼ Explain Clearly

We are calculating mean and standard deviation using FULL data,
including test data → this causes data leakage.

```
scaler = StandardScaler()
X_scaled_wrong = scaler.fit_transform(X)
```

▼ STEP 4A: Train-Test Split

```
X_train_w, X_test_w, y_train_w, y_test_w = train_test_split(
    X_scaled_wrong, y, test_size=0.3, random_state=42
)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split  # ✅ FIX
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Student dataset
data = {
    'Age': [18, 19, 20, 21, 22, 19, 20, 21, 18, 22],
    'Attendance': [75, 85, 60, 90, 95, 88, 65, 92, 70, 98],
    'Study_Hours': [2, 4, 1, 5, 6, 4, 2, 5, 3, 6],
    'Result': [0, 1, 0, 1, 1, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

X = df[['Age', 'Attendance', 'Study_Hours']]
y = df['Result']

# WRONG way (Data Leakage)
scaler = StandardScaler()
X_scaled_wrong = scaler.fit_transform(X)
```

```

X_train_w, X_test_w, y_train_w, y_test_w = train_test_split(
    X_scaled_wrong, y, test_size=0.3, random_state=42
)

model_wrong = LogisticRegression()
model_wrong.fit(X_train_w, y_train_w)

y_pred_w = model_wrong.predict(X_test_w)
print("Accuracy with Data Leakage:", accuracy_score(y_test_w, y_pred_w))

```

Accuracy with Data Leakage: 1.0

In this code data leakage is cooucred due to

fit_transform(X) computes mean and standard deviation using the ENTIRE dataset

This includes future test data

So, test data information leaks into training

👉 This is data leakage

✓ CORRECT CODE (No Data Leakage)

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# -----
# Student Dataset
# -----
data = {
    'Age': [18, 19, 20, 21, 22, 19, 20, 21, 18, 22],
    'Attendance': [75, 85, 60, 90, 95, 88, 65, 92, 70, 98],
    'Study_Hours': [2, 4, 1, 5, 6, 4, 2, 5, 3, 6],
    'Result': [0, 1, 0, 1, 1, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

X = df[['Age', 'Attendance', 'Study_Hours']]
y = df['Result']

# -----
# ✓ CORRECT: Split first
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# -----
# ✓ Scaling after split
# -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # ✓ fit on train
X_test_scaled = scaler.transform(X_test)      # ✓ transform test

# -----
# Model Training & Evaluation
# -----
model = LogisticRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
print("Accuracy without Data Leakage:", accuracy_score(y_test, y_pred))

```

Accuracy without Data Leakage: 0.6666666666666666

1 Underfitting (model too simple)

2 Overfitting (model too complex)

Problem:

Predict Final Marks based on Study Hours

	Study_Hours	Marks
0	1	35
1	2	40
2	3	50
3	4	60
4	5	65
5	6	70
6	7	80
7	8	85
8	9	90
9	10	95

Double-click (or enter) to edit

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
```

```
# Create DataFrame
data = {
    'Study_Hours': [1,2,3,4,5,6,7,8,9,10],
    'Marks': [35,40,50,60,65,70,80,85,90,95]
}

df = pd.DataFrame(data)
print(df)
```

	Study_Hours	Marks
0	1	35
1	2	40
2	3	50
3	4	60
4	5	65
5	6	70
6	7	80
7	8	85
8	9	90
9	10	95

Each row = one student

Study_Hours → feature (X)

Marks → target (y)

STEP 2: Separate Feature & Target

```
X = df[['Study_Hours']] # must be 2D for sklearn
y = df['Marks']
```

STEP 3: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

CASE 1: UNDERFITTING (Model Too Simple)

👉 Linear Regression on a non-linear pattern

```
# Linear Regression (Too Simple)
```

```

# Linear Regression (see sample)
model_underfit = LinearRegression()
model_underfit.fit(X_train, y_train)

# Predictions
y_train_pred = model_underfit.predict(X_train)
y_test_pred = model_underfit.predict(X_test)

# Errors
train_error = mean_squared_error(y_train, y_train_pred)
test_error = mean_squared_error(y_test, y_test_pred)

print("UNDERFITTING MODEL")
print("Training Error:", train_error)
print("Testing Error :", test_error)

```

UNDERFITTING MODEL
 Training Error: 3.818953323903816
 Testing Error : 6.290232983694369

First: What is this error?

The error shown is Mean Squared Error (MSE).

 Formula:

$$MSE = \frac{1}{n} \sum (y_{actual} - y_{predicted})^2$$

-  Lower error = better model
-  Higher error = poor model

What this result tells us

1 Training Error ≈ 3.82

- Model is **not fitting training data well**
- Predictions are far from actual marks
- Model is **too simple**

2 Testing Error ≈ 6.29

- Error on unseen data is even **higher**
- Model fails to generalize
- This confirms **poor learning**

Model Type	Training Error	Testing Error
Underfitting	High	High
Overfitting	Very Low	High
Good Fit	Low	Low

✓ Your result matches **UNDERFITTING pattern perfectly** ✓

Why underfitting happened in this code?

You used this model:

```
python
model_underfit = LinearRegression()
```

But the data pattern is **non-linear**:

- Marks increase slowly at first
- Then increase faster with study hours

 A **straight line** cannot capture this curve.

The training error is high, which means the model cannot even learn the training data

properly.

The testing error is also high, which means the model cannot generalize. Therefore, this is an underfitting model.

Double-click (or enter) to edit

```
1  # Polynomial Regression with Degree = 2
   # Student Marks Dataset

   import pandas as pd
   import numpy as np
   import matplotlib.pyplot as plt

   from sklearn.model_selection import train_test_split
   from sklearn.preprocessing import PolynomialFeatures
   from sklearn.linear_model import LinearRegression
   from sklearn.metrics import mean_squared_error

   # -----
   # Step 1: Create Student Marks DataFrame
   # -----
   data = {
       'Study_Hours': [1,2,3,4,5,6,7,8,9,10],
       'Marks': [35,40,50,60,65,70,80,85,90,95]
   }

   df = pd.DataFrame(data)

   # -----
   # Step 2: Separate Feature and Target
   # -----
   X = df[['Study_Hours']] # Feature must be 2D
   y = df['Marks']         # Target

   # -----
   # Step 3: Train-Test Split
   # -----
   X_train, X_test, y_train, y_test = train_test_split(
       X, y, test_size=0.3, random_state=42
   )

   # -----
   # Step 4: Polynomial Feature Transformation (Degree = 2)
   # -----
   poly = PolynomialFeatures(degree=2)

   X_train_poly = poly.fit_transform(X_train)
   X_test_poly = poly.transform(X_test)

   # -----
   # Step 5: Train the Model
   # -----
   model = LinearRegression()
   model.fit(X_train_poly, y_train)

   # -----
   # Step 6: Predictions
   # -----
   y_train_pred = model.predict(X_train_poly)
   y_test_pred = model.predict(X_test_poly)

   # -----
   # Step 7: Model Evaluation
   # -----
   train_error = mean_squared_error(y_train, y_train_pred)
   test_error = mean_squared_error(y_test, y_test_pred)
```

```

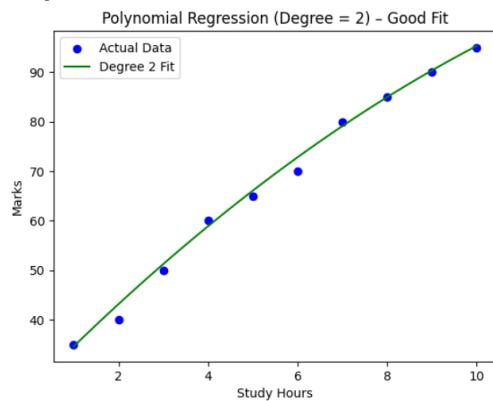
print("Polynomial Regression (Degree = 2)")
print("Training Error:", train_error)
print("Testing Error :", test_error)

# -----
# Step 8: Visualization
# -----
X_curve = pd.DataFrame({'Study_Hours': np.linspace(1, 10, 100)})
X_curve_poly = poly.transform(X_curve)
y_curve = model.predict(X_curve_poly)

plt.scatter(df['Study_Hours'], df['Marks'], color='blue', label='Actual Data')
plt.plot(X_curve['Study_Hours'], y_curve, color='green', label='Degree 2 Fit')
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Polynomial Regression (Degree = 2) - Good Fit")
plt.legend()
plt.show()

```

Polynomial Regression (Degree = 2)
 Training Error: 0.7099610782759139
 Testing Error : 5.863944776449135



Compare WITH the Underfitting Model

✗ Underfitting (Linear Regression)

javascript

Training Error $\approx$ 3.82
 Testing Error $\approx$ 6.29

✓ Polynomial Degree = 2

javascript

Training Error $\approx$ 0.71 **↓ BIG DROP**
 Testing Error $\approx$ 5.86 **↓ DROP**

👉 You must compare models RELATIVELY, not absolutely.

Model	Train Error	Test Error	Verdict
Linear (Underfit)	High	High	Bad
Poly Degree 2	Lower	Lower	Better
Poly Degree 9	Very Low	Very High	Overfit

✓ Testing error did NOT increase

✓ It decreased from 6.29 → 5.86

So the model improved.

⚠ Why is Testing Error Still Not Very Low?

This is normal and EXPECTED because:

1 Very small dataset

- Only 10 students
- Test set has 3 samples
- Error becomes unstable

2 No noise-free perfect pattern

- Marks are **not mathematically perfect**
- Real-world data always has error

3 Degree 2 is a balanced compromise

- Enough complexity to reduce bias
- Not too complex to overfit

Accuracy → overall correctness

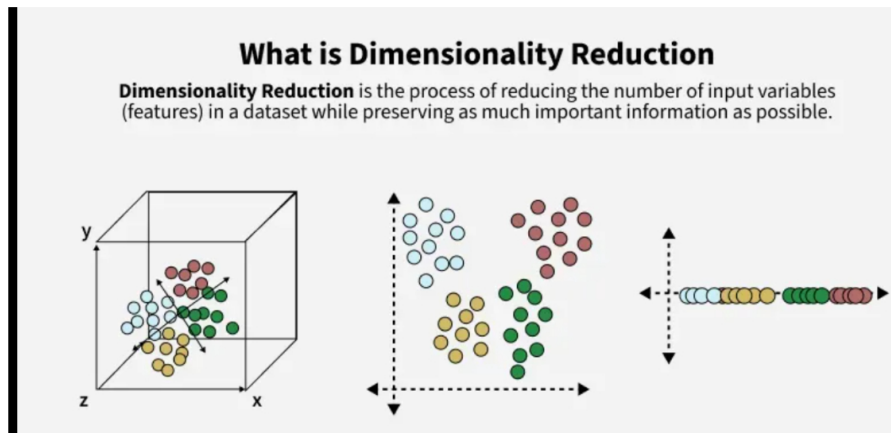
Precision → how many predicted Pass are actually Pass

Recall → how many actual Pass students were identified

Confusion Matrix → TP, FP, FN, TN

✓ Principal Component Analysis (PCA) is a dimensionality reduction technique.

It transform high-dimensional data into a smaller number of dimensions called principal components and keeps important information in the data.



What is Scikit-learn?

Scikit-learn is a Python machine learning library used for:

Classification

Regression

Clustering

Dimensionality reduction

Model evaluation

Why Scikit-learn is Popular?

- ✓ Simple and consistent API
- ✓ Large collection of ML algorithms
- ✓ Easy preprocessing tools
- ✓ Excellent documentation
- ✓ Industry + academia standard

✓ Machine Learning Workflow using Scikit-learn

Dataset
↓
Preprocessing
↓

↓
Train-Test Split
↓
Model Selection
↓
Training (fit)
↓
Prediction
↓
Evaluation

▼ Important Modules in Scikit-learn

1. Dataset Module

```
[ ] from sklearn import datasets
```

▼ Used for:

Built-in datasets (Iris, Digits)

Example:

```
[ ] from sklearn.datasets import load_iris  
data = load_iris()
```

▼ 2. Model Selection

```
[ ] Start coding or generate with AI.
```

▼ Purpose:

Split data into training and testing

Example:

```
[ ] from sklearn.model_selection import train_test_split
```

▼ 3. Preprocessing Module

```
[ ] from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
```

▼ Used for:

Feature scaling

Encoding categorical data

Example:

```
[ ] scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

```
-----  
NameError                                Traceback (most recent call last)  
/tmp/ipython-input-2462607029.py in <cell line: 0>()  
      1 scaler = StandardScaler()  
----> 2 X_scaled = scaler.fit_transform(X)  
  
NameError: name 'X' is not defined
```

Supervised Learning Algorithms

▼ Classification

```
[ ] from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.svm import SVC
```

▼ Regression

```
[1] from sklearn.linear_model import LinearRegression
    from sklearn.ensemble import RandomForestRegressor
```

▼ Unsupervised Learning

```
[1] from sklearn.cluster import KMeans
    from sklearn.decomposition import PCA
```

Used for:

Clustering

Dimensionality reduction

▼ Model Evaluation

Double-click (or enter) to edit

```
[1] from sklearn.metrics import accuracy_score, confusion_matrix, mean_squared_error
```

Used for:

Checking model performance

▼ Scikit-learn API Concept (VERY IMPORTANT)

All sklearn models follow the same 3 steps:

```
[1] model = Algorithm()
    model.fit(X_train, y_train)
    model.predict(X_test)
```

This is called Estimator API

Students Marks using Scikit-learn

Problem

Predict Pass / Fail based on marks.

▼ Step 1: Dataset

```
[1] import pandas as pd

    data = {
        'Study_Hours': [2,4,6,8,10],
        'Marks': [40,55,70,85,95]
    }

    df = pd.DataFrame(data)
```

▼ Step 2: Create Target

```
[1] df['Result'] = df['Marks'].apply(lambda x: 1 if x >= 50 else 0)
```

▼ Step 3: Split Data

```
[1] from sklearn.model_selection import train_test_split

    X = df[['Study_Hours']]
    y = df['Result']
```

```
y = ur[ result ]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

Step 4: Train Model

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

Step 5: Prediction & Evaluation

```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

Advantages of Scikit-learn

- ✓ Easy to learn
- ✓ Consistent syntax
- ✓ Fast prototyping
- ✓ Strong community
- ✓ Works well with small-medium datasets

Limitations of Scikit-learn

- ✗ Not suitable for very large deep learning models
- ✗ No GPU support
- ✗ Limited for image/text deep learning
- 👉 For deep learning → TensorFlow / PyTorch

Feature	Scikit-learn	TensorFlow
Type	ML	Deep Learning
GPU	✗ No	✓ Yes
Best for	Tabular data	Images/Text
Learning curve	Easy	Steep

Scikit-learn: Python library for machine learning

fit(): Trains the model

predict(): Predicts output

train_test_split: Prevents overfitting

StandardScaler: Feature normalization

Colab paid products - Cancel contracts here

 Variables  Terminal

